MODUL PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBYEK

MODUL 9
Implementasi UML Class Diagram ke Program Java

Disusun oleh:

Nirwana Haidar Hari, S.Pd., M.Kom.

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
PROGRAM STUDI DI LUAR KAMPUS UTAMA SUMENEP
PENS PSDKU SUMENEP

2026

MODUL PRAKTIKUM P9
Implementasi UML Class Diagram ke Program Java

Mata Kuliah: Praktikum Pemrograman Berorientasi Obyek
Pertemuan: 9
Topik: Implementasi UML Class Diagram
Model Pembelajaran: Project Based Learning

1. Tujuan Praktikum

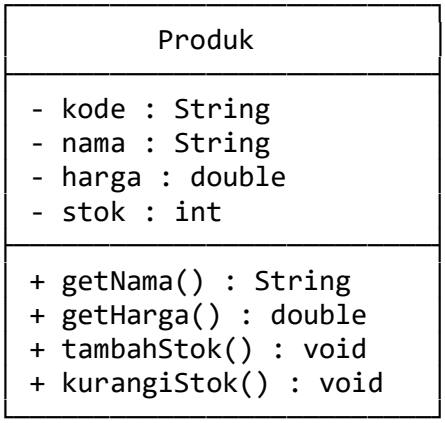
Setelah menyelesaikan praktikum, mahasiswa mampu:

- membaca UML Class Diagram sederhana;
- menerjemahkan class, attribute, method, dan relasi ke dalam kode Java;
- mengimplementasikan encapsulation dan relasi antarobject;
- menjaga kesesuaian antara rancangan UML dan source code;
- mengembangkan bagian proyek berdasarkan desain yang telah dibuat.

2. Dasar Teori Singkat

Class Diagram digunakan untuk menggambarkan struktur suatu sistem berorientasi object.

Komponen utama:



Keterangan:

Simbol	Arti
+	public
-	private
#	protected

Representasi dalam Java:

```
public class Produk {
```

```
    private String kode;
    private String nama;
    private double harga;
    private int stok;

    public String getNama() {
        return nama;
    }

    public double getHarga() {
        return harga;
    }

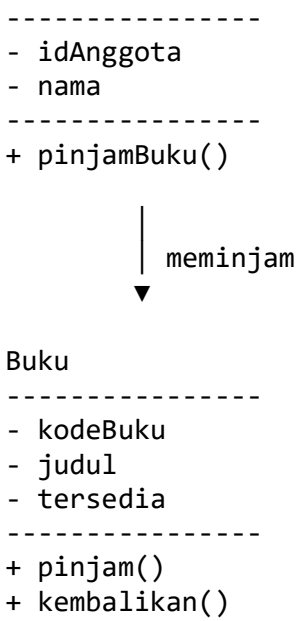
    public void tambahStok(int jumlah) {
        if (jumlah > 0) {
            stok += jumlah;
        }
    }
}
```

Prinsip penting:

> *UML merupakan rancangan, sedangkan source code merupakan implementasinya. Keduanya harus konsisten.*

3. Contoh Kasus

Perhatikan rancangan sistem perpustakaan berikut:



Salah satu implementasi:

```
public class Buku {

    private String kodeBuku;
    private String judul;
    private boolean tersedia;

    public Buku(String kodeBuku, String judul) {
        this.kodeBuku = kodeBuku;
        this.judul = judul;
        this.tersedia = true;
    }

    public void pinjam() {
        if (tersedia) {
            tersedia = false;
        }
    }

    public void kembalikan() {
        tersedia = true;
    }
}
```

4. Kegiatan Praktikum

Langkah 1 — Analisis UML

Perhatikan Class Diagram proyek kelompok.

Identifikasi:

- class;
- attribute;
- method;
- visibility;
- relasi antarclass;
- multiplicity jika tersedia.

Langkah 2 — Buat Struktur Class

Setiap class pada UML dibuat menjadi file Java.

Contoh:

```
src/
├── Produk.java
├── Pelanggan.java
├── Transaksi.java
```

```
└── DetailTransaksi.java
```

Langkah 3 — Implementasikan Attribute

Pastikan attribute sesuai dengan UML.

Contoh:

```
private String nama;
private double harga;
private int stok;
```

Langkah 4 — Implementasikan Constructor dan Method

Tambahkan:

- constructor;
- getter jika diperlukan;
- method sesuai tanggung jawab class;
- validasi sederhana.

Langkah 5 — Implementasikan Relasi Antarobject

Contoh:

```
public class DetailTransaksi {

    private Produk produk;
    private int jumlah;

    public DetailTransaksi(Produk produk, int jumlah) {
        this.produk = produk;
        this.jumlah = jumlah;
    }
}
```

Langkah 6 — Pengujian

Buat class:

```
Main.java
```

untuk menguji interaksi object.

Contoh:

```
public class Main {
```

```
        public static void main(String[] args) {

            Produk produk =
                new Produk("P001", "Keyboard", 250000, 10);

            produk.tambahStok(5);

        }
    }
```

5. Tugas Project Based Learning

Lanjutkan **proyek kelompok** yang telah dirancang pada pertemuan sebelumnya.

Implementasikan minimal:

- **4 class utama**;
- attribute dengan visibility yang tepat;
- constructor;
- method sesuai responsibility;
- minimal **2 relasi antarobject**;
- encapsulation;
- validasi state sederhana.

Contoh struktur:

```
Project/
├── model/
│   ├── Produk.java
│   ├── Pelanggan.java
│   ├── Transaksi.java
│   └── DetailTransaksi.java
│
└── Main.java
```

6. Ketentuan

Mahasiswa **tidak diperbolehkan hanya membuat class berisi attribute + getter/setter**.

Setiap class harus mempunyai:

responsibility yang jelas dan behavior yang relevan dengan kasus proyek.

Contoh kurang tepat:

```
setStok(-100);
```

Lebih tepat:

```
kurangiStok(2);
```

karena object bertanggung jawab menjaga state-nya.

7. Luaran

Setiap Mahasiswa mengumpulkan:

1. UML Class Diagram versi terbaru.
2. Source code Java.
3. Screenshot hasil program.
4. Penjelasan singkat hubungan UML dengan implementasi.

Format folder:

```
P9_NamaMHS_NamaProject/
├── UML/
├── SourceCode/
└── Screenshot/
```

8. Penilaian

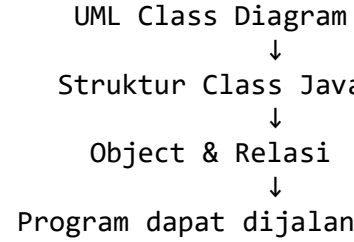
Aspek	Bobot
Kesesuaian UML dengan source code	30%
Implementasi class dan object	25%
Implementasi relasi antarobject	20%
Encapsulation dan responsibility	15%
Program dapat dijalankan dan dijelaskan	10%
Total	**100%**

9. Pertanyaan Evaluasi

Sebelum meninggalkan praktikum, setiap kelompok harus dapat menjawab:

1. Apa perbedaan **class pada UML** dan **object pada program**?
2. Bagaimana association pada UML diterapkan dalam Java?
3. Mengapa attribute sebaiknya tidak seluruhnya `public`?
4. Apakah source code proyek sudah konsisten dengan UML?
5. Class mana yang memiliki responsibility paling besar dalam proyek Anda?

Target Pertemuan 9



Fokus utama: mahasiswa tidak sekadar membuat kode, tetapi mampu menunjukkan bahwa **implementasi Java berasal dari rancangan object-oriented yang jelas**.